

Retail Analytics

Complete PySpark Bronze → Silver Curation Code

Databricks notebook documentation and implementation for processing the six Bronze tables into detailed curated Silver tables. The pipeline performs cleansing, standardization, deduplication, validation, enrichment, derived business attributes, Delta MERGE processing, data-quality checks, and final validation.

Target Silver Model

Bronze Source	Silver Target	Grain / Purpose
customer	dim_customer	1 row per customer
product	dim_product	1 row per product
orders	fact_orders	Order-product transaction detail
store_sales	fact_store_sales	Store transaction detail
inventory	fact_inventory	Store-product-date inventory snapshot
payment	fact_payment	Payment transaction detail
orders + store_sales + customer + product + payment	fact_sales_detail	Unified enriched sales transaction detail

Transformation Flow

Bronze raw data → schema validation → null/invalid handling → string and data type standardization → duplicate removal → business rules → referential integrity → customer/product/payment enrichment → derived measures → date/time enrichment → audit columns → Delta MERGE → Silver data-quality validation.

Important Business Assumption

The provided Bronze model does not contain an explicit discount column. The notebook therefore derives an estimated discount as gross_product_value minus order/sale amount, floored at zero. Tax is estimated from the transaction amount and product tax rate. If the source system defines order_amount or sale_amount differently, these formulas should be adjusted.

Complete Databricks PySpark Notebook Code

```
# =====
# RETAIL ANALYTICS
# BRONZE -> SILVER CURATION PIPELINE
# Databricks PySpark Notebook
# =====

from pyspark.sql import functions as F
from pyspark.sql import types as T
from pyspark.sql.window import Window
from delta.tables import DeltaTable

# =====
# 1. DATABASE CONFIGURATION
# =====

CATALOG = "retail_catalog"

BRONZE_SCHEMA = "bronze_retail"
SILVER_SCHEMA = "silver_retail"

BRONZE = f"{CATALOG}.{BRONZE_SCHEMA}"
SILVER = f"{CATALOG}.{SILVER_SCHEMA}"

spark.sql(f"""
CREATE SCHEMA IF NOT EXISTS {CATALOG}.{SILVER_SCHEMA}
""")

# =====
# 2. READ BRONZE TABLES
# =====

df_customer_bronze = spark.table(f"{BRONZE}.customer")
df_product_bronze = spark.table(f"{BRONZE}.product")
df_orders_bronze = spark.table(f"{BRONZE}.orders")
df_store_sales_bronze = spark.table(f"{BRONZE}.store_sales")
df_inventory_bronze = spark.table(f"{BRONZE}.inventory")
df_payment_bronze = spark.table(f"{BRONZE}.payment")

print("Bronze tables successfully loaded.")
```

```

# =====
# 3. BRONZE RECORD COUNTS
# =====

bronze_counts = {
    "customer": df_customer_bronze.count(),
    "product": df_product_bronze.count(),
    "orders": df_orders_bronze.count(),
    "store_sales": df_store_sales_bronze.count(),
    "inventory": df_inventory_bronze.count(),
    "payment": df_payment_bronze.count()
}

for table_name, count in bronze_counts.items():
    print(f"{table_name:20} : {count:,}")

# =====
# 4. COMMON CLEANING FUNCTIONS
# =====

def clean_string(column_name):
    """
    Trim leading/trailing spaces and convert empty strings to NULL.
    """
    return F.when(
        F.trim(F.col(column_name)) == "",
        None
    ).otherwise(F.trim(F.col(column_name)))

def upper_string(column_name):
    """
    Trim and convert string values to uppercase.
    """
    return F.upper(F.trim(F.col(column_name)))

def lower_string(column_name):
    """
    Trim and convert string values to lowercase.
    """
    return F.lower(F.trim(F.col(column_name)))

# =====
# 5. CUSTOMER CURATION
# Bronze : customer
# Silver : dim_customer
# Grain : 1 row per customer

```

```

# =====

df_customer = (
    df_customer_bronze
    .withColumn("customer_id", clean_string("customer_id"))
    .withColumn("customer_name", clean_string("customer_name"))
    .withColumn("email", lower_string("email"))
    .withColumn("phone", clean_string("phone"))
    .withColumn("city", clean_string("city"))
    .withColumn("state", upper_string("state"))
    .withColumn("status", upper_string("status"))

    .filter(F.col("customer_id").isNull())

    .dropDuplicates(["customer_id"])

    .withColumn(
        "customer_tenure_days",
        F.datediff(F.current_date(), F.col("registration_date"))
    )

    .withColumn(
        "is_active",
        F.when(
            F.col("status").isin("ACTIVE", "A"),
            F.lit(True)
        ).otherwise(F.lit(False))
    )

    .withColumn(
        "silver_load_timestamp",
        F.current_timestamp()
    )
)

df_customer_silver = df_customer.select(
    "customer_id",
    "customer_name",
    "email",
    "phone",
    "city",
    "state",
    "registration_date",
    "status",
    "customer_tenure_days",
    "is_active",
    "_source_file",
    "_ingestion_timestamp",
    "silver_load_timestamp"
)

display(df_customer_silver)

# =====
# 6. PRODUCT CURATION
# Bronze : product
# Silver : dim_product
# Grain : 1 row per product
# =====

df_product = (
    df_product_bronze
    .withColumn("product_id", clean_string("product_id"))
    .withColumn("product_name", clean_string("product_name"))
    .withColumn("category", upper_string("category"))
    .withColumn("subcategory", upper_string("subcategory"))
    .withColumn("brand", upper_string("brand"))
    .withColumn("status", upper_string("status"))

    .withColumn(
        "unit_price",
        F.when(F.col("unit_price") >= 0, F.col("unit_price"))
        .otherwise(F.lit(None))
    )

    .withColumn(
        "tax_rate_pct",
        F.when(
            (F.col("tax_rate_pct") >= 0) &
            (F.col("tax_rate_pct") <= 100),
            F.col("tax_rate_pct")
        ).otherwise(F.lit(None))
    )

    .filter(F.col("product_id").isNull())

    .dropDuplicates(["product_id"])

    .withColumn(
        "price_band",
        F.when(F.col("unit_price") < 100, "LOW")
        .when(F.col("unit_price") < 500, "MEDIUM")
        .when(F.col("unit_price") < 1000, "HIGH")
        .otherwise("PREMIUM")
    )

    .withColumn(
        "is_active",
        F.when(
            F.col("status").isin("ACTIVE", "A"),
            F.lit(True)
        ).otherwise(F.lit(False))
    )
)

```

```

        .withColumn(
            "silver_load_timestamp",
            F.current_timestamp()
        )
    )

df_product_silver = df_product.select(
    "product_id",
    "product_name",
    "category",
    "subcategory",
    "unit_price",
    "tax_rate_pct",
    "brand",
    "status",
    "price_band",
    "is_active",
    "silver_load_timestamp"
)

display(df_product_silver)

# =====
# 7. ORDERS CURATION
# Bronze : orders
# Silver : fact_orders
# Grain : 1 row per order-product transaction
# =====

df_orders = (
    df_orders_bronze
    .withColumn("order_id", clean_string("order_id"))
    .withColumn("customer_id", clean_string("customer_id"))
    .withColumn("product_id", clean_string("product_id"))
    .withColumn("channel", upper_string("channel"))
    .withColumn("order_status", upper_string("order_status"))

    .withColumn(
        "quantity",
        F.when(F.col("quantity") > 0, F.col("quantity"))
        .otherwise(F.lit(None))
    )

    .withColumn(
        "order_amount",
        F.when(F.col("order_amount") >= 0, F.col("order_amount"))
        .otherwise(F.lit(None))
    )

    .filter(
        F.col("order_id").isNotNull() &
        F.col("product_id").isNotNull() &
        F.col("order_timestamp").isNotNull()
    )

    .dropDuplicates([
        "order_id",
        "product_id",
        "order_timestamp"
    ])
)

df_orders_silver = (
    df_orders.alias("o")
    .join(
        df_product_silver.alias("p"),
        F.col("o.product_id") == F.col("p.product_id"),
        "left"
    )
    .select(
        F.col("o.order_id"),
        F.col("o.customer_id"),
        F.col("o.product_id"),
        F.col("o.channel"),
        F.col("o.order_timestamp"),
        F.col("o.quantity"),
        F.col("o.order_amount"),
        F.col("o.order_status"),

        F.col("p.product_name"),
        F.col("p.category"),
        F.col("p.subcategory"),
        F.col("p.brand"),
        F.col("p.unit_price"),
        F.col("p.tax_rate_pct"),
        F.col("p.price_band"),

        (
            F.col("o.quantity") * F.col("p.unit_price")
        ).cast("decimal(18,2)").alias(
            "gross_product_value"
        ),

        F.greatest(
            (
                F.col("o.quantity") * F.col("p.unit_price")
            ) - F.col("o.order_amount"),
            F.lit(0)
        ).cast("decimal(18,2)").alias(
            "estimated_discount_amount"
        ),

        (

```

```

        F.col("o.order_amount") *
        F.coalesce(F.col("p.tax_rate_pct"), F.lit(0)) / 100
    ).cast("decimal(18,2)").alias(
        "estimated_tax_amount"
    ),

    F.to_date("o.order_timestamp").alias("order_date"),
    F.year("o.order_timestamp").alias("order_year"),
    F.month("o.order_timestamp").alias("order_month"),
    F.quarter("o.order_timestamp").alias("order_quarter"),
    F.dayofweek("o.order_timestamp").alias("order_day_of_week"),
    F.hour("o.order_timestamp").alias("order_hour"),

    F.current_timestamp().alias("silver_load_timestamp")
)
)

display(df_orders_silver)

```

```

# =====
# 8. STORE SALES CURATION
# Bronze : store_sales
# Silver : fact_store_sales
# Grain : 1 row per store sale
# =====

df_store_sales = (
    df_store_sales_bronze
    .withColumn("sale_id", clean_string("sale_id"))
    .withColumn("store_id", clean_string("store_id"))
    .withColumn("customer_id", clean_string("customer_id"))
    .withColumn("product_id", clean_string("product_id"))
    .withColumn("payment_method", upper_string("payment_method"))

    .withColumn(
        "quantity",
        F.when(F.col("quantity") > 0, F.col("quantity"))
        .otherwise(F.lit(None))
    )

    .withColumn(
        "sale_amount",
        F.when(F.col("sale_amount") >= 0, F.col("sale_amount"))
        .otherwise(F.lit(None))
    )

    .filter(
        F.col("sale_id").isNotNull() &
        F.col("product_id").isNotNull() &
        F.col("sale_timestamp").isNotNull()
    )

    .dropDuplicates(["sale_id"])
)

df_store_sales_silver = (
    df_store_sales.alias("s")
    .join(
        df_product_silver.alias("p"),
        F.col("s.product_id") == F.col("p.product_id"),
        "left"
    )
    .select(
        F.col("s.sale_id"),
        F.col("s.store_id"),
        F.col("s.customer_id"),
        F.col("s.product_id"),
        F.col("s.sale_timestamp"),
        F.col("s.quantity"),
        F.col("s.sale_amount"),
        F.col("s.payment_method"),

        F.col("p.product_name"),
        F.col("p.category"),
        F.col("p.subcategory"),
        F.col("p.brand"),
        F.col("p.unit_price"),
        F.col("p.tax_rate_pct"),
        F.col("p.price_band"),

        (
            F.col("s.quantity") * F.col("p.unit_price")
        ).cast("decimal(18,2)").alias(
            "gross_product_value"
        ),

        F.greatest(
            (
                F.col("s.quantity") * F.col("p.unit_price")
            ) - F.col("s.sale_amount"),
            F.lit(0)
        ).cast("decimal(18,2)").alias(
            "estimated_discount_amount"
        ),

        (
            F.col("s.sale_amount") *
            F.coalesce(F.col("p.tax_rate_pct"), F.lit(0)) / 100
        ).cast("decimal(18,2)").alias(
            "estimated_tax_amount"
        ),

        F.to_date("s.sale_timestamp").alias("sale_date"),
        F.year("s.sale_timestamp").alias("sale_year"),
        F.month("s.sale_timestamp").alias("sale_month"),
        F.quarter("s.sale_timestamp").alias("sale_quarter"),
        F.dayofweek("s.sale_timestamp").alias("sale_day_of_week"),
        F.hour("s.sale_timestamp").alias("sale_hour"),

        F.current_timestamp().alias("silver_load_timestamp")
    )
)

display(df_store_sales_silver)

# =====
# 9. INVENTORY CURATION
# Bronze : inventory
# Silver : fact_inventory
# Grain : 1 row per store-product-date
# =====

```

```

df_inventory_silver = (
    df_inventory_bronze
    .withColumn("inventory_id", clean_string("inventory_id"))
    .withColumn("store_id", clean_string("store_id"))
    .withColumn("product_id", clean_string("product_id"))

    .withColumn(
        "available_qty",
        F.when(F.col("available_qty") >= 0, F.col("available_qty"))
        .otherwise(F.lit(0))
    )

    .withColumn(
        "reorder_level",
        F.when(F.col("reorder_level") >= 0, F.col("reorder_level"))
        .otherwise(F.lit(0))
    )

    .withColumn(
        "safety_stock",
        F.when(F.col("safety_stock") >= 0, F.col("safety_stock"))
        .otherwise(F.lit(0))
    )

    .filter(
        F.col("inventory_id").isNotNull() &
        F.col("store_id").isNotNull() &
        F.col("product_id").isNotNull() &
        F.col("inventory_date").isNotNull()
    )

    .dropDuplicates(["inventory_id"])

    .join(
        df_product_silver.select(
            "product_id",
            "product_name",
            "category",
            "subcategory",
            "brand",
            "unit_price"
        ),
        "product_id",
        "left"
    )

    .withColumn(
        "inventory_value",
        (
            F.col("available_qty") *
            F.col("unit_price")
        ).cast("decimal(18,2)")
    )

    .withColumn(
        "stock_status",
        F.when(
            F.col("available_qty") == 0,
            "OUT_OF_STOCK"
        )
        .when(
            F.col("available_qty") <= F.col("safety_stock"),
            "CRITICAL"
        )
        .when(
            F.col("available_qty") <= F.col("reorder_level"),
            "REORDER"
        )
        .otherwise("HEALTHY")
    )

    .withColumn(
        "reorder_required",
        F.when(
            F.col("available_qty") <= F.col("reorder_level"),
            F.lit(True)
        ).otherwise(F.lit(False))
    )

    .withColumn(
        "inventory_year",
        F.year("inventory_date")
    )

    .withColumn(
        "inventory_month",
        F.month("inventory_date")
    )

    .withColumn(
        "silver_load_timestamp",
        F.current_timestamp()
    )
)

display(df_inventory_silver)

```

```

# =====
# 10. PAYMENT CURATION
# Bronze : payment
# Silver : fact_payment
# Grain : 1 row per payment transaction

```

```

# =====

df_payment_silver = (
    df_payment_bronze
    .withColumn("payment_id", clean_string("payment_id"))
    .withColumn("order_id", clean_string("order_id"))
    .withColumn("customer_id", clean_string("customer_id"))
    .withColumn("payment_method", upper_string("payment_method"))
    .withColumn("payment_status", upper_string("payment_status"))

    .withColumn(
        "payment_amount",
        F.when(
            F.col("payment_amount") >= 0,
            F.col("payment_amount")
        ).otherwise(F.lit(None))
    )

    .filter(
        F.col("payment_id").isNotNull() &
        F.col("payment_timestamp").isNotNull()
    )

    .dropDuplicates(["payment_id"])

    .withColumn(
        "payment_success_flag",
        F.when(
            F.col("payment_status").isin(
                "SUCCESS",
                "COMPLETED",
                "PAID"
            ),
            F.lit(True)
        ).otherwise(F.lit(False))
    )

    .withColumn(
        "payment_date",
        F.to_date("payment_timestamp")
    )

    .withColumn(
        "payment_year",
        F.year("payment_timestamp")
    )

    .withColumn(
        "payment_month",
        F.month("payment_timestamp")
    )

    .withColumn(
        "silver_load_timestamp",
        F.current_timestamp()
    )
)

display(df_payment_silver)

# =====
# 11. REFERENTIAL INTEGRITY CHECKS
# =====

valid_customers = (
    df_customer_silver
    .select("customer_id")
    .distinct()
)

valid_products = (
    df_product_silver
    .select("product_id")
    .distinct()
)

invalid_order_customers = (
    df_orders_silver
    .join(valid_customers, "customer_id", "left_anti")
)

invalid_order_products = (
    df_orders_silver
    .join(valid_products, "product_id", "left_anti")
)

print(
    "Orders with invalid customers:",
    invalid_order_customers.count()
)

print(
    "Orders with invalid products:",
    invalid_order_products.count()
)

# =====
# 12. ONLINE SALES DETAIL
# Used to build unified fact_sales_detail
# =====

customer_enrichment = (

```

```

df_customer_silver
.select(
  "customer_id",
  "customer_name",
  "customer_tenure_days"
)
.withColumn(
  "customer_lifecycle",
  F.when(
    F.col("customer_tenure_days") < 90,
    "NEW"
  )
  .when(
    F.col("customer_tenure_days") < 365,
    "GROWING"
  )
  .otherwise("LOYAL")
)
)

df_online_detail = (
  df_orders_silver.alias("o")

  .join(
    customer_enrichment.alias("c"),
    "customer_id",
    "left"
  )

  .join(
    df_payment_silver
    .groupBy("order_id")
    .agg(
      F.max("payment_status").alias("payment_status"),
      F.max("payment_method").alias("payment_method")
    ),
    "order_id",
    "left"
  )

  .select(
    F.col("order_id").alias("transaction_id"),
    F.lit("ONLINE").alias("sales_channel"),

    F.col("customer_id"),
    F.col("product_id"),
    F.lit(None).cast("string").alias("store_id"),

    F.col("order_date").alias("transaction_date"),
    F.col("order_timestamp").alias("transaction_timestamp"),

    F.col("quantity"),
    F.col("order_amount").alias("sales_amount"),

    F.col("gross_product_value"),
    F.col("estimated_discount_amount").alias("discount_amount"),
    F.col("estimated_tax_amount").alias("tax_amount"),

    F.col("payment_method"),
    F.col("payment_status"),

    F.col("customer_name"),
    F.col("customer_lifecycle"),

    F.col("product_name"),
    F.col("category"),
    F.col("subcategory"),
    F.col("brand"),

    F.col("channel"),
    F.col("order_status").alias("transaction_status"),

    F.col("order_year").alias("transaction_year"),
    F.col("order_month").alias("transaction_month"),
    F.col("order_quarter").alias("transaction_quarter"),
    F.col("order_day_of_week"),
    F.col("order_hour"),

    F.current_timestamp().alias("silver_load_timestamp")
  )
)

# =====
# 13. STORE SALES DETAIL
# =====

df_store_detail = (
  df_store_sales_silver.alias("s")

  .join(
    customer_enrichment.alias("c"),
    "customer_id",
    "left"
  )

  .select(
    F.col("sale_id").alias("transaction_id"),
    F.lit("STORE").alias("sales_channel"),

    F.col("customer_id"),
    F.col("product_id"),
    F.col("store_id"),

```

```

    F.col("sale_date").alias("transaction_date"),
    F.col("sale_timestamp").alias("transaction_timestamp"),

    F.col("quantity"),
    F.col("sale_amount").alias("sales_amount"),

    F.col("gross_product_value"),
    F.col("estimated_discount_amount").alias("discount_amount"),
    F.col("estimated_tax_amount").alias("tax_amount"),

    F.col("payment_method"),
    F.lit(None).cast("string").alias("payment_status"),

    F.col("customer_name"),
    F.col("customer_lifecycle"),

    F.col("product_name"),
    F.col("category"),
    F.col("subcategory"),
    F.col("brand"),

    F.lit("STORE").alias("channel"),
    F.lit("COMPLETED").alias("transaction_status"),

    F.col("sale_year").alias("transaction_year"),
    F.col("sale_month").alias("transaction_month"),
    F.col("sale_quarter").alias("transaction_quarter"),
    F.col("sale_day_of_week").alias("order_day_of_week"),
    F.col("sale_hour").alias("order_hour"),

    F.current_timestamp().alias("silver_load_timestamp")
  )
)

# =====
# 14. UNION ONLINE + STORE SALES
# =====

df_sales_detail = (
  df_online_detail
    .unionByName(
      df_store_detail,
      allowMissingColumns=True
    )
)

# =====
# 15. CALENDAR / DATE ENRICHMENT
# =====

df_sales_detail = (
  df_sales_detail

    .withColumn(
      "transaction_day",
      F.dayofmonth("transaction_date")
    )

    .withColumn(
      "transaction_week",
      F.weekofyear("transaction_date")
    )

    .withColumn(
      "transaction_day_name",
      F.date_format(
        "transaction_date",
        "EEEE"
      )
    )

    .withColumn(
      "transaction_month_name",
      F.date_format(
        "transaction_date",
        "MMMM"
      )
    )

    .withColumn(
      "transaction_year_month",
      F.date_format(
        "transaction_date",
        "yyyy-MM"
      )
    )

    .withColumn(
      "weekend_flag",
      F.when(
        F.dayofweek("transaction_date").isin(1, 7),
        True
      ).otherwise(False)
    )

    .withColumn(
      "sales_amount",
      F.col("sales_amount").cast("decimal(18,2)")
    )

    .withColumn(
      "net_sales_amount",

```

```

        (
            F.col("sales_amount") -
            F.coalesce(
                F.col("discount_amount"),
                F.lit(0)
            )
        ).cast("decimal(18,2)")
    )

    .dropDuplicates([
        "transaction_id",
        "sales_channel"
    ])
)

display(df_sales_detail)

# =====
# 16. CREATE SILVER TABLES
# =====

# Dimension Customer
spark.sql(f"""
CREATE TABLE IF NOT EXISTS {SILVER}.dim_customer
USING DELTA
AS
SELECT *
FROM (
    SELECT
        CAST(NULL AS STRING) AS customer_id,
        CAST(NULL AS STRING) AS customer_name,
        CAST(NULL AS STRING) AS email,
        CAST(NULL AS STRING) AS phone,
        CAST(NULL AS STRING) AS city,
        CAST(NULL AS STRING) AS state,
        CAST(NULL AS DATE) AS registration_date,
        CAST(NULL AS STRING) AS status,
        CAST(NULL AS BIGINT) AS customer_tenure_days,
        CAST(NULL AS BOOLEAN) AS is_active,
        CAST(NULL AS STRING) AS _source_file,
        CAST(NULL AS TIMESTAMP) AS _ingestion_timestamp,
        CAST(NULL AS TIMESTAMP) AS silver_load_timestamp
    WHERE FALSE
)
""")

# Dimension Product
spark.sql(f"""
CREATE TABLE IF NOT EXISTS {SILVER}.dim_product
USING DELTA
AS
SELECT *
FROM (
    SELECT
        CAST(NULL AS STRING) AS product_id,
        CAST(NULL AS STRING) AS product_name,
        CAST(NULL AS STRING) AS category,
        CAST(NULL AS STRING) AS subcategory,
        CAST(NULL AS DECIMAL(18,2)) AS unit_price,
        CAST(NULL AS DECIMAL(5,2)) AS tax_rate_pct,
        CAST(NULL AS STRING) AS brand,
        CAST(NULL AS STRING) AS status,
        CAST(NULL AS STRING) AS price_band,
        CAST(NULL AS BOOLEAN) AS is_active,
        CAST(NULL AS TIMESTAMP) AS silver_load_timestamp
    WHERE FALSE
)
""")

# Fact Orders
spark.sql(f"""
CREATE TABLE IF NOT EXISTS {SILVER}.fact_orders
USING DELTA
AS
SELECT *
FROM (
    SELECT
        CAST(NULL AS STRING) AS order_id,
        CAST(NULL AS STRING) AS customer_id,
        CAST(NULL AS STRING) AS product_id,
        CAST(NULL AS STRING) AS channel,
        CAST(NULL AS TIMESTAMP) AS order_timestamp,
        CAST(NULL AS INT) AS quantity,
        CAST(NULL AS DECIMAL(18,2)) AS order_amount,
        CAST(NULL AS STRING) AS order_status,
        CAST(NULL AS STRING) AS product_name,
        CAST(NULL AS STRING) AS category,
        CAST(NULL AS STRING) AS subcategory,
        CAST(NULL AS STRING) AS brand,
        CAST(NULL AS DECIMAL(18,2)) AS unit_price,
        CAST(NULL AS DECIMAL(5,2)) AS tax_rate_pct,
        CAST(NULL AS STRING) AS price_band,
        CAST(NULL AS DECIMAL(18,2)) AS gross_product_value,
        CAST(NULL AS DECIMAL(18,2)) AS estimated_discount_amount,
        CAST(NULL AS DECIMAL(18,2)) AS estimated_tax_amount,
        CAST(NULL AS DATE) AS order_date,
        CAST(NULL AS INT) AS order_year,
        CAST(NULL AS INT) AS order_month,
        CAST(NULL AS INT) AS order_quarter,
        CAST(NULL AS INT) AS order_day_of_week,
        CAST(NULL AS INT) AS order_hour,

```

```

        CAST(NULL AS TIMESTAMP) AS silver_load_timestamp
    WHERE FALSE
)
)"""

# Fact Store Sales
spark.sql(f"""
CREATE TABLE IF NOT EXISTS {SILVER}.fact_store_sales
USING DELTA
AS
SELECT *
FROM (
    SELECT
        CAST(NULL AS STRING) AS sale_id,
        CAST(NULL AS STRING) AS store_id,
        CAST(NULL AS STRING) AS customer_id,
        CAST(NULL AS STRING) AS product_id,
        CAST(NULL AS TIMESTAMP) AS sale_timestamp,
        CAST(NULL AS INT) AS quantity,
        CAST(NULL AS DECIMAL(18,2)) AS sale_amount,
        CAST(NULL AS STRING) AS payment_method,
        CAST(NULL AS STRING) AS product_name,
        CAST(NULL AS STRING) AS category,
        CAST(NULL AS STRING) AS subcategory,
        CAST(NULL AS STRING) AS brand,
        CAST(NULL AS DECIMAL(18,2)) AS unit_price,
        CAST(NULL AS DECIMAL(5,2)) AS tax_rate_pct,
        CAST(NULL AS STRING) AS price_band,
        CAST(NULL AS DECIMAL(18,2)) AS gross_product_value,
        CAST(NULL AS DECIMAL(18,2)) AS estimated_discount_amount,
        CAST(NULL AS DECIMAL(18,2)) AS estimated_tax_amount,
        CAST(NULL AS DATE) AS sale_date,
        CAST(NULL AS INT) AS sale_year,
        CAST(NULL AS INT) AS sale_month,
        CAST(NULL AS INT) AS sale_quarter,
        CAST(NULL AS INT) AS sale_day_of_week,
        CAST(NULL AS INT) AS sale_hour,
        CAST(NULL AS TIMESTAMP) AS silver_load_timestamp
    WHERE FALSE
)
)"""

# Fact Inventory
spark.sql(f"""
CREATE TABLE IF NOT EXISTS {SILVER}.fact_inventory
USING DELTA
AS
SELECT *
FROM (
    SELECT
        CAST(NULL AS STRING) AS inventory_id,
        CAST(NULL AS STRING) AS store_id,
        CAST(NULL AS STRING) AS product_id,
        CAST(NULL AS DATE) AS inventory_date,
        CAST(NULL AS INT) AS available_qty,
        CAST(NULL AS INT) AS reorder_level,
        CAST(NULL AS INT) AS safety_stock,
        CAST(NULL AS STRING) AS product_name,
        CAST(NULL AS STRING) AS category,
        CAST(NULL AS STRING) AS subcategory,
        CAST(NULL AS STRING) AS brand,
        CAST(NULL AS DECIMAL(18,2)) AS unit_price,
        CAST(NULL AS DECIMAL(18,2)) AS inventory_value,
        CAST(NULL AS STRING) AS stock_status,
        CAST(NULL AS BOOLEAN) AS reorder_required,
        CAST(NULL AS INT) AS inventory_year,
        CAST(NULL AS INT) AS inventory_month,
        CAST(NULL AS TIMESTAMP) AS silver_load_timestamp
    WHERE FALSE
)
)"""

# Fact Payment
spark.sql(f"""
CREATE TABLE IF NOT EXISTS {SILVER}.fact_payment
USING DELTA
AS
SELECT *
FROM (
    SELECT
        CAST(NULL AS STRING) AS payment_id,
        CAST(NULL AS STRING) AS order_id,
        CAST(NULL AS STRING) AS customer_id,
        CAST(NULL AS DECIMAL(18,2)) AS payment_amount,
        CAST(NULL AS STRING) AS payment_method,
        CAST(NULL AS STRING) AS payment_status,
        CAST(NULL AS TIMESTAMP) AS payment_timestamp,
        CAST(NULL AS BOOLEAN) AS payment_success_flag,
        CAST(NULL AS DATE) AS payment_date,
        CAST(NULL AS INT) AS payment_year,
        CAST(NULL AS INT) AS payment_month,
        CAST(NULL AS TIMESTAMP) AS silver_load_timestamp
    WHERE FALSE
)
)"""

# Fact Sales Detail
spark.sql(f"""
CREATE TABLE IF NOT EXISTS {SILVER}.fact_sales_detail
USING DELTA

```

```

AS
SELECT *
FROM (
    SELECT
        CAST(NULL AS STRING) AS transaction_id,
        CAST(NULL AS STRING) AS sales_channel,
        CAST(NULL AS STRING) AS customer_id,
        CAST(NULL AS STRING) AS product_id,
        CAST(NULL AS STRING) AS store_id,
        CAST(NULL AS DATE) AS transaction_date,
        CAST(NULL AS TIMESTAMP) AS transaction_timestamp,
        CAST(NULL AS INT) AS quantity,
        CAST(NULL AS DECIMAL(18,2)) AS sales_amount,
        CAST(NULL AS DECIMAL(18,2)) AS gross_product_value,
        CAST(NULL AS DECIMAL(18,2)) AS discount_amount,
        CAST(NULL AS DECIMAL(18,2)) AS tax_amount,
        CAST(NULL AS DECIMAL(18,2)) AS net_sales_amount,
        CAST(NULL AS STRING) AS payment_method,
        CAST(NULL AS STRING) AS payment_status,
        CAST(NULL AS STRING) AS customer_name,
        CAST(NULL AS STRING) AS customer_lifecycle,
        CAST(NULL AS STRING) AS product_name,
        CAST(NULL AS STRING) AS category,
        CAST(NULL AS STRING) AS subcategory,
        CAST(NULL AS STRING) AS brand,
        CAST(NULL AS STRING) AS channel,
        CAST(NULL AS STRING) AS transaction_status,
        CAST(NULL AS INT) AS transaction_year,
        CAST(NULL AS INT) AS transaction_month,
        CAST(NULL AS INT) AS transaction_quarter,
        CAST(NULL AS INT) AS order_day_of_week,
        CAST(NULL AS INT) AS order_hour,
        CAST(NULL AS INT) AS transaction_day,
        CAST(NULL AS INT) AS transaction_week,
        CAST(NULL AS STRING) AS transaction_day_name,
        CAST(NULL AS STRING) AS transaction_month_name,
        CAST(NULL AS STRING) AS transaction_year_month,
        CAST(NULL AS BOOLEAN) AS weekend_flag,
        CAST(NULL AS TIMESTAMP) AS silver_load_timestamp
    WHERE FALSE
)
)
)

# =====
# 17. MERGE DIM_CUSTOMER
# =====

target_customer = DeltaTable.forName(
    spark,
    f"{SILVER}.dim_customer"
)

(
    target_customer.alias("t")
    .merge(
        df_customer_silver.alias("s"),
        "t.customer_id = s.customer_id"
    )
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)

# =====
# 18. MERGE DIM_PRODUCT
# =====

target_product = DeltaTable.forName(
    spark,
    f"{SILVER}.dim_product"
)

(
    target_product.alias("t")
    .merge(
        df_product_silver.alias("s"),
        "t.product_id = s.product_id"
    )
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)

# =====
# 19. MERGE FACT_ORDERS
# =====

target_orders = DeltaTable.forName(
    spark,
    f"{SILVER}.fact_orders"
)

(
    target_orders.alias("t")
    .merge(
        df_orders_silver.alias("s"),
        """
        t.order_id = s.order_id
        AND t.product_id = s.product_id
        AND t.order_timestamp = s.order_timestamp
        """
    )
)

```

```

        .whenMatchedUpdateAll()
        .whenNotMatchedInsertAll()
        .execute()
    )

# =====
# 20. MERGE FACT_STORE_SALES
# =====

target_store_sales = DeltaTable.forName(
    spark,
    f"{SILVER}.fact_store_sales"
)

(
    target_store_sales.alias("t")
    .merge(
        df_store_sales_silver.alias("s"),
        "t.sale_id = s.sale_id"
    )
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)

# =====
# 21. MERGE FACT_INVENTORY
# =====

target_inventory = DeltaTable.forName(
    spark,
    f"{SILVER}.fact_inventory"
)

(
    target_inventory.alias("t")
    .merge(
        df_inventory_silver.alias("s"),
        "t.inventory_id = s.inventory_id"
    )
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)

# =====
# 22. MERGE FACT_PAYMENT
# =====

target_payment = DeltaTable.forName(
    spark,
    f"{SILVER}.fact_payment"
)

(
    target_payment.alias("t")
    .merge(
        df_payment_silver.alias("s"),
        "t.payment_id = s.payment_id"
    )
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)

# =====
# 23. MERGE FACT_SALES_DETAIL
# =====

target_sales_detail = DeltaTable.forName(
    spark,
    f"{SILVER}.fact_sales_detail"
)

(
    target_sales_detail.alias("t")
    .merge(
        df_sales_detail.alias("s"),
        """
        t.transaction_id = s.transaction_id
        AND t.sales_channel = s.sales_channel
        """
    )
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)

# =====
# 24. SILVER DATA QUALITY CHECKS
# =====

print("CUSTOMER QUALITY")

spark.sql(f"""
SELECT
    COUNT(*) AS total_records,
    COUNT(DISTINCT customer_id) AS distinct_customers,
    SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END)
    AS null_customer_id,

```

```

        SUM(CASE WHEN email IS NULL THEN 1 ELSE 0 END)
        AS null_email
FROM {SILVER}.dim_customer
""").show()

print("PRODUCT QUALITY")

spark.sql(f"""
SELECT
    COUNT(*) AS total_records,
    COUNT(DISTINCT product_id) AS distinct_products,
    SUM(CASE WHEN product_id IS NULL THEN 1 ELSE 0 END)
    AS null_product_id,
    SUM(CASE WHEN unit_price < 0 THEN 1 ELSE 0 END)
    AS invalid_price
FROM {SILVER}.dim_product
""").show()

print("ORDER QUALITY")

spark.sql(f"""
SELECT
    COUNT(*) AS total_records,
    COUNT(DISTINCT order_id) AS distinct_orders,
    SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END)
    AS null_customer_id,
    SUM(CASE WHEN product_id IS NULL THEN 1 ELSE 0 END)
    AS null_product_id,
    SUM(CASE WHEN quantity <= 0 THEN 1 ELSE 0 END)
    AS invalid_quantity
FROM {SILVER}.fact_orders
""").show()

print("INVENTORY QUALITY")

spark.sql(f"""
SELECT
    COUNT(*) AS total_records,
    SUM(CASE WHEN available_qty < 0 THEN 1 ELSE 0 END)
    AS negative_inventory,
    SUM(CASE WHEN reorder_required = TRUE THEN 1 ELSE 0 END)
    AS reorder_records,
    SUM(
        CASE WHEN stock_status = 'OUT_OF_STOCK'
        THEN 1 ELSE 0 END
    ) AS out_of_stock_records
FROM {SILVER}.fact_inventory
""").show()

print("PAYMENT QUALITY")

spark.sql(f"""
SELECT
    COUNT(*) AS total_records,
    SUM(
        CASE WHEN payment_success_flag = TRUE
        THEN 1 ELSE 0 END
    ) AS successful_payments,
    SUM(
        CASE WHEN payment_success_flag = FALSE
        THEN 1 ELSE 0 END
    ) AS failed_payments
FROM {SILVER}.fact_payment
""").show()

# =====
# 25. FINAL SILVER TABLE COUNTS
# =====

silver_tables = [
    "dim_customer",
    "dim_product",
    "fact_orders",
    "fact_store_sales",
    "fact_inventory",
    "fact_payment",
    "fact_sales_detail"
]

for table in silver_tables:
    count = spark.table(
        f"{SILVER}.{table}"
    ).count()

    print(
        f"{SILVER}.{table}:25 : {count:,"}
    )

# =====
# 26. OPTIONAL OPTIMIZE
# =====

for table in silver_tables:
    spark.sql(
        f"OPTIMIZE {SILVER}.{table}"
    )

print("Silver curation pipeline completed successfully.")

```